Homework 5

DATA 624

Kevin Havis

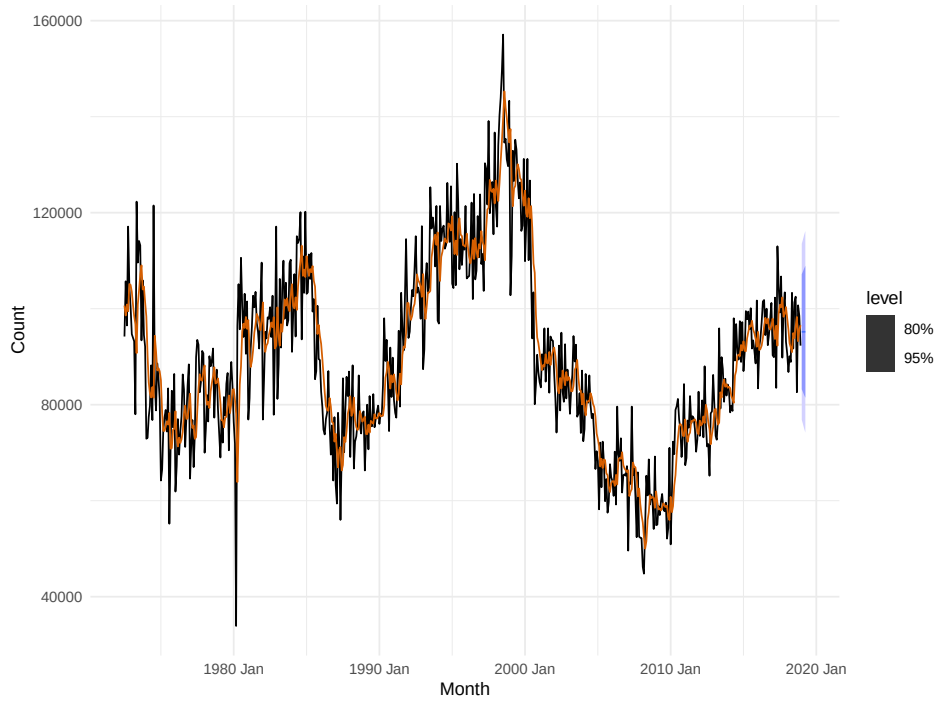
October 5, 2025

8.1

Consider the the number of pigs slaughtered in Victoria, available in the aus_livestock dataset.

Use the ETS() function to estimate the equivalent model for simple exponential smoothing. Find the optimal values of and 0, and generate forecasts for the next four months.

```
# Filter data
pigs <- aus_livestock %>%
  filter(Animal == "Pigs", State == "Victoria") %>%
  select(Month, Count)

# Fit the model
fit <- pigs %>%
  model(ETS(Count ~ error("A") + trend("N") + season("N")))

# Forecast new observations
fc <- fit |>
  forecast(h = 4)

# Plot
fc |>
  autoplot(pigs) +
  geom_line(aes(y = .fitted), col="#D55E00",
            data = augment(fit))
```




```
# Print the components
print(fit |> tidy())
```

```
# A tibble: 2 x 3
  .model                                                    term    estimate
  <chr>                                                     <chr>      <dbl>
1 "ETS(Count ~ error(\"A\") + trend(\"N\") + season(\"N\"))" alpha      0.322
2 "ETS(Count ~ error(\"A\") + trend(\"N\") + season(\"N\"))" l[0]   100647.
```

Compute a 95% prediction interval for the first forecast using $\hat{y} \pm 1.96s$ where s is the standard deviation of the residuals. Compare your interval with the interval produced by R.

Our manual prediction, using $\hat{y}_{T+1|T} \pm 1.96 s_1$, gives an interval of $[73,984, 110,615]$, compared to the model's $[76,854, 113,518]$. This is quite close, especially considering our manual calculation is quite simple and is only performing a one step forecast (which is also why it is slightly lower; we have less residuals to worry about in a single step).

```
# Get the last value of the data
last_pigs <- pigs %>%
  tail(1) %>%
  pull(Count)
```

```r
# Calculate 95% CI interval of residual distribution
interval <- c(
  (last_pigs - 1.96 * sd(residuals(fit)$.resid)),
  (last_pigs + 1.96 * sd(residuals(fit)$.resid))
)
print(interval)
```

```
[1]  73984.46 110615.54
```

```r
# Pull the innovation residuals
model_interval <- fc |>
  filter(row_number() == 1) |>
  hilo(level = 95) |>
  unpack_hilo('95%')

tibble(
  method = c("Manual", "Model", "Diff"),
  lower = c(interval[1], model_interval$`95%_lower`,
            interval[1] - model_interval$`95%_lower`),
  upper = c(interval[2], model_interval$`95%_upper`,
            interval[2] - model_interval$`95%_upper`),
)
```

```
# A tibble: 3 x 3
  method  lower   upper
  <chr>   <dbl>   <dbl>
1 Manual 73984. 110616.
2 Model  76855. 113518.
3 Diff   -2870.  -2903.
```

8.5

Data set global_economy contains the annual Exports from many countries. Select one country to analyse.

- Plot the Exports series and discuss the main features of the data.
- Use an ETS(A,N,N) model to forecast the series, and plot the forecasts.
- Compute the RMSE values for the training data.
- Compare the results to those from an ETS(A,A,N) model. (Remember that the trended model is using one more parameter than the simpler model.) Discuss the merits of the two forecasting methods for this data set.

- Compare the forecasts from both methods. Which do you think is best?
- Calculate a 95% prediction interval for the first forecast for each model, using the RMSE values and assuming normal errors. Compare your intervals with those produced using R.

Canadian export data has as steady trend upward, with a very large peak in the later half of the series. We can see small seasonal bumps throughout.

Reviewing the RMSE of the `ETS(A,N,N)` and `ETS(A,A,N)` models, we get slightly lower RMSE using the `ETS(A,A,N)` model. Despite this, the `ETS(A,A,N)` does require a trend component for minimally improved RMSE, so we may prefer the simpler model (without trend component).

The forecast of ANN model is more stable, with a smaller interval and a flat trend (as we've excluded the trend component). The AAN model is capturing the very recent downward trend, but given the wider interval and the overall trajectory of the time series, as well as the aforementioned simplicity, I would prefer the ANN model for forecasting.

Comparing the intervals computed with R vs manually, they are again very close to each other, with the ANN model being slightly narrower.

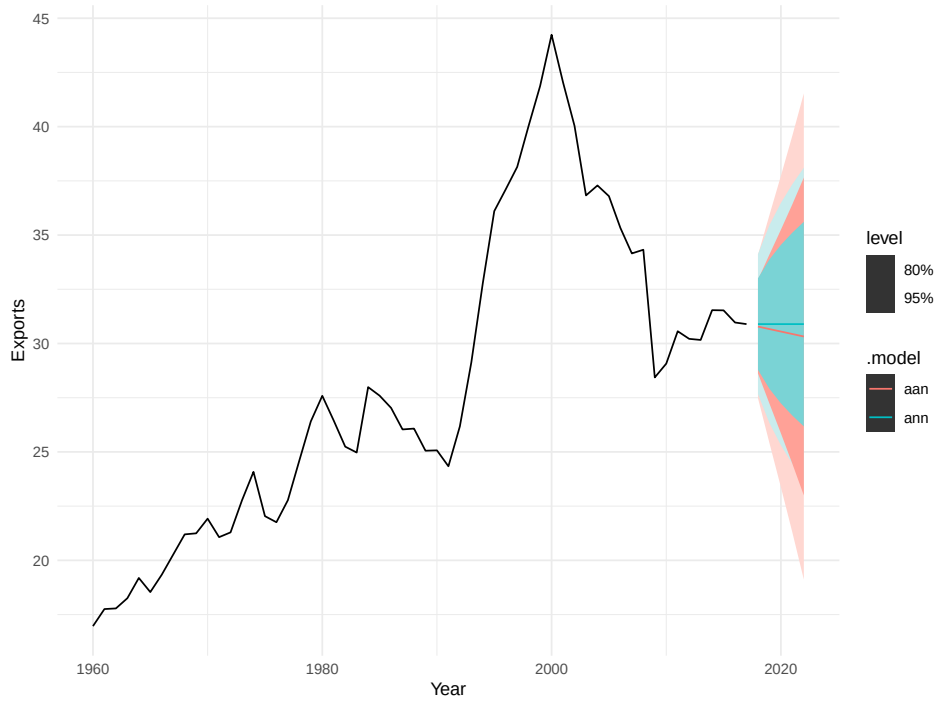
```
# Filter data for a specific country
canada <- global_economy %>%
  filter(Country == "Canada") %>%
  select(Year, Exports)

# Plot
autoplot(canada, Exports) +
  labs(title = "Annual Exports from Canada",
       y = "Exports (in million USD)",
       x = "Year")
```


```
# Fit models
canada_fit  <- canada %>%
  model(ann = ETS(Exports ~ error("A") + trend("N") + season("N")),
        aan = ETS(Exports ~ error("A") + trend("A") + season("N"))
        )

# Get RMSE
canada_fit %>%
  accuracy() %>%
  select(.model, RMSE)
```

```
# A tibble: 2 x 2
  .model  RMSE
  <chr>  <dbl>
1 ann     1.62
2 aan     1.63
```

```
# Get forecasts
canada_fc <- canada_fit %>%
  forecast(h = "5 years")
```

```r
# Plot
canada_fc %>%
  autoplot(canada)
```




```r
# Get residuals for each model
resid_ann <- canada_fit %>%
  select(ann) %>%
  residuals() %>%
  pull(.resid)

resid_aan <- canada_fit %>%
  select(aan) %>%
  residuals() %>%
  pull(.resid)

# Manually calculate 95% prediction interval for ANN
last_canada <- tail(canada$Exports, 1)
interval_ann <- c(
  last_canada - 1.96 * sd(resid_ann),
  last_canada + 1.96 * sd(resid_ann)
)
```

```r
# Manually calculate 95% prediction interval for AAN
interval_aan <- c(
  last_canada - 1.96 * sd(resid_aan),
  last_canada + 1.96 * sd(resid_aan)
)

# Get model intervals
fc_ann <- canada_fit %>%
  select(ann) %>%
  forecast(h = 1) %>%
  hilo(level = 95) %>%
  unpack_hilo(`95%`)

fc_aan <- canada_fit %>%
  select(aan) %>%
  forecast(h = 1) %>%
  hilo(level = 95) %>%
  unpack_hilo(`95%`)

# ANN comparison
tibble(
  method = c("Manual", "Model", "Diff"),
  lower = c(interval_ann[1], fc_ann$`95%_lower`,
            interval_ann[1] - fc_ann$`95%_lower`),
  upper = c(interval_ann[2], fc_ann$`95%_upper`,
            interval_ann[2] - fc_ann$`95%_upper`)
)
```

```
# A tibble: 3 x 3
  method   lower   upper
  <chr>    <dbl>   <dbl>
1 Manual 27.7    34.1
2 Model  27.7    34.1
3 Diff    0.0638 -0.0638
```

```r
# AAN comparison
tibble(
  method = c("Manual", "Model", "Diff"),
  lower = c(interval_aan[1], fc_aan$`95%_lower`,
            interval_aan[1] - fc_aan$`95%_lower`),
  upper = c(interval_aan[2], fc_aan$`95%_upper`,
```

```
            interval_aan[2] - fc_aan$`95%_upper`)
)
```

```
# A tibble: 3 x 3
  method  lower   upper
  <chr>   <dbl>   <dbl>
1 Manual 27.7   34.1
2 Model  27.5   34.1
3 Diff    0.202  0.0262
```

```
tibble(
  method = c("Manual", "Model", "Diff"),
  lower = c(interval_ann[1], fc_ann$`95%_lower`,
            interval_ann[1] - fc_ann$`95%_lower`),
  upper = c(interval_ann[2], fc_ann$`95%_upper`,
            interval_ann[2] - fc_ann$`95%_upper`)
) %>%
  knitr::kable(digits = 2)
```

Table 1: Comparison of manual vs model prediction intervals for ANN model

method	lower	upper
Manual	27.73	34.06
Model	27.67	34.12
Diff	0.06	-0.06

```
tibble(
  method = c("Manual", "Model", "Diff"),
  lower = c(interval_aan[1], fc_aan$`95%_lower`,
            interval_aan[1] - fc_aan$`95%_lower`),
  upper = c(interval_aan[2], fc_aan$`95%_upper`,
            interval_aan[2] - fc_aan$`95%_upper`)
) %>%
  knitr::kable(digits = 2)
```

Table 2: Comparison of manual vs model prediction intervals for AAN model

method	lower	upper
Manual	27.68	34.11
Model	27.48	34.08
Diff	0.20	0.03

8.6

Forecast the Chinese GDP from the global_economy data set using an ETS model. Experiment with the various options in the ETS() function to see how much the forecasts change with damped trend, or with a Box-Cox transformation. Try to develop an intuition of what each is doing to the forecasts.

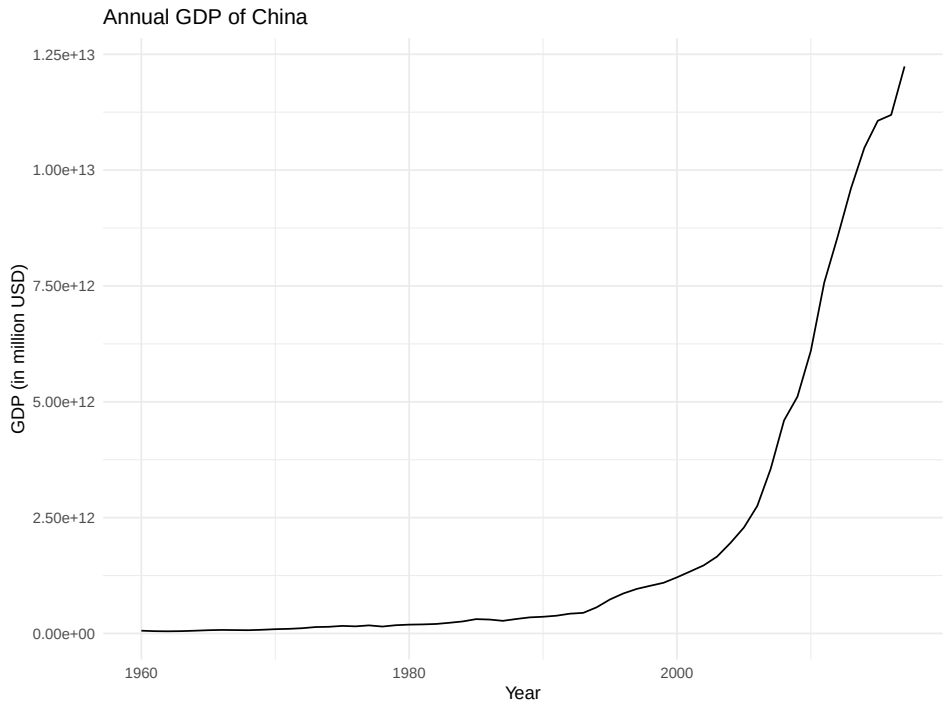
[Hint: use a relatively large value of h when forecasting, so you can clearly see the differences between the various options when plotting the forecasts.]

The parameterized framework of the ETS model is very convenient for experimenting. It is interesting to see how responsive the model can be when introducing the various components.

This data does not exhibit much seasonality and as such the seasonal component is not very useful. Experimenting with the trend component shows us how dampening may be effective, and how the trend component affects the series.

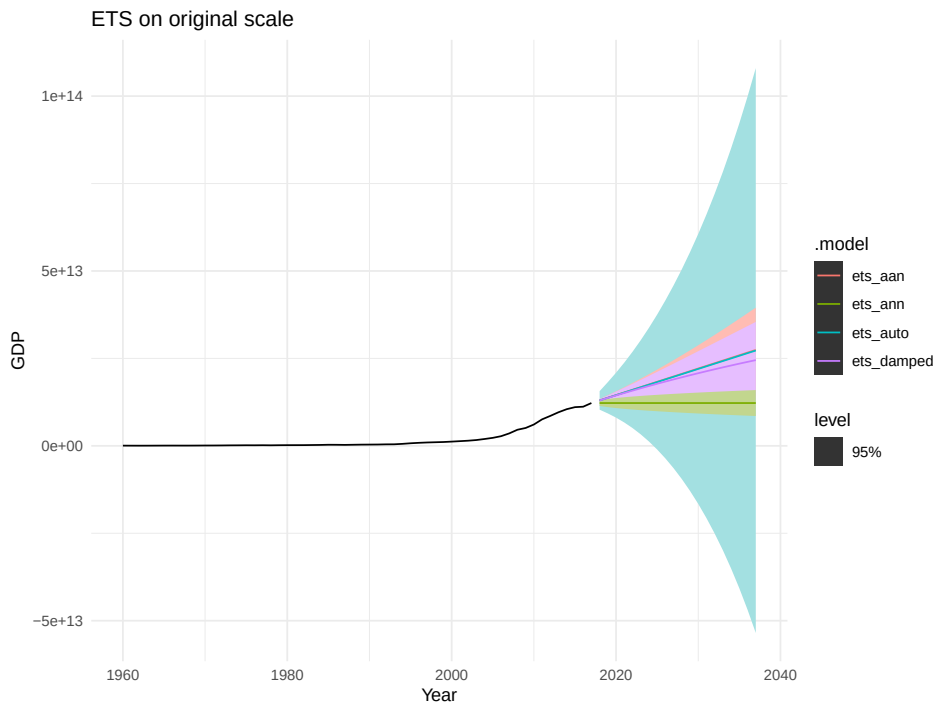
Particularly interesting were the BoxCox and similar transformations. I was surprised to see how similar a MMN model looked compared to Box-Cox, until I recalled that the multiplicative nature of the transformation essential is scaling the data, which is what a Box-Cox transformation does as well.

```
# Load data
china_gdp <- global_economy %>%
  filter(Country == "China") %>%
  select(Year, GDP)

# Plot
autoplot(china_gdp, GDP) +
  labs(title = "Annual GDP of China",
       y = "GDP (in million USD)",
       x = "Year")
```

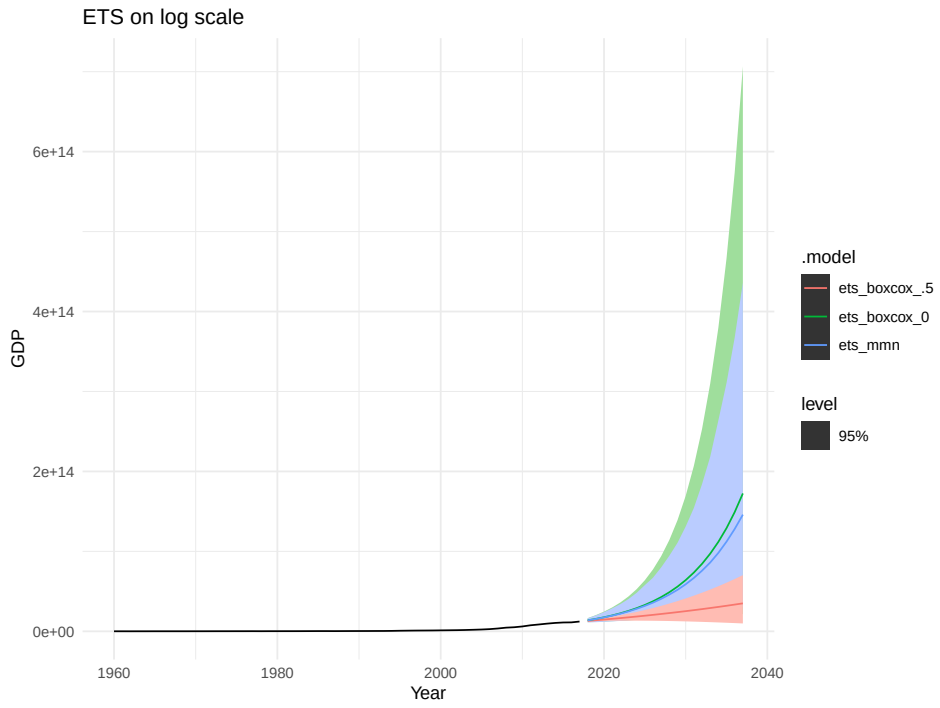



```
# Fit models
china_fit <- china_gdp %>%
  model(
    ets_auto = ETS(GDP),
    ets_ann = ETS(GDP ~ error("A") + trend("N") + season("N")),
    ets_aan = ETS(GDP ~ error("A") + trend("A") + season("N")),
    ets_damped = ETS(GDP ~ error("A") + trend("Ad") + season("N"))
    #ets_aan_bc = ETS(box_cox(GDP, lambda = 0) ~ error("A") + trend("A") + season("N"))
     )

# Separate the log ones for better visualization; different scale

china_fit_log <- china_gdp |>
  model(ets_boxcox_0 = ETS(box_cox(GDP, lambda = 0) ~ error("A") + trend("A") + season("N"))
        ets_boxcox_.5 = ETS(box_cox(GDP, lambda = 0.5) ~ error("A") + trend("A") + season("N"
        ets_mmn = ETS(GDP ~ error("M") + trend("M") + season("N"))
        )

china_fc <- china_fit |>
  forecast(h=20)

china_fc |>
```

```
  autoplot(china_gdp, level = 95) + labs(title="ETS on original scale")
```




```
# Create the log models
china_fc_log <- china_fit_log |>
  forecast(h=20)

china_fc_log |>
  autoplot(china_gdp, level = 95) + labs(title="ETS on log scale")
```



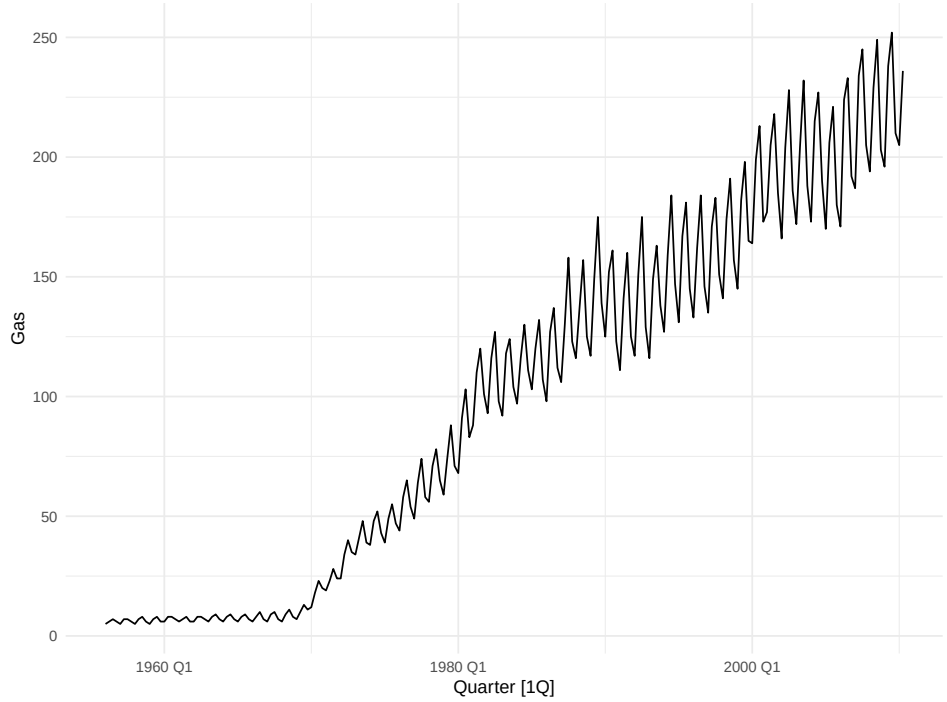
8.7

Find an ETS model for the Gas data from aus_production and forecast the next few years. Why is multiplicative seasonality necessary here? Experiment with making the trend damped. Does it improve the forecasts?

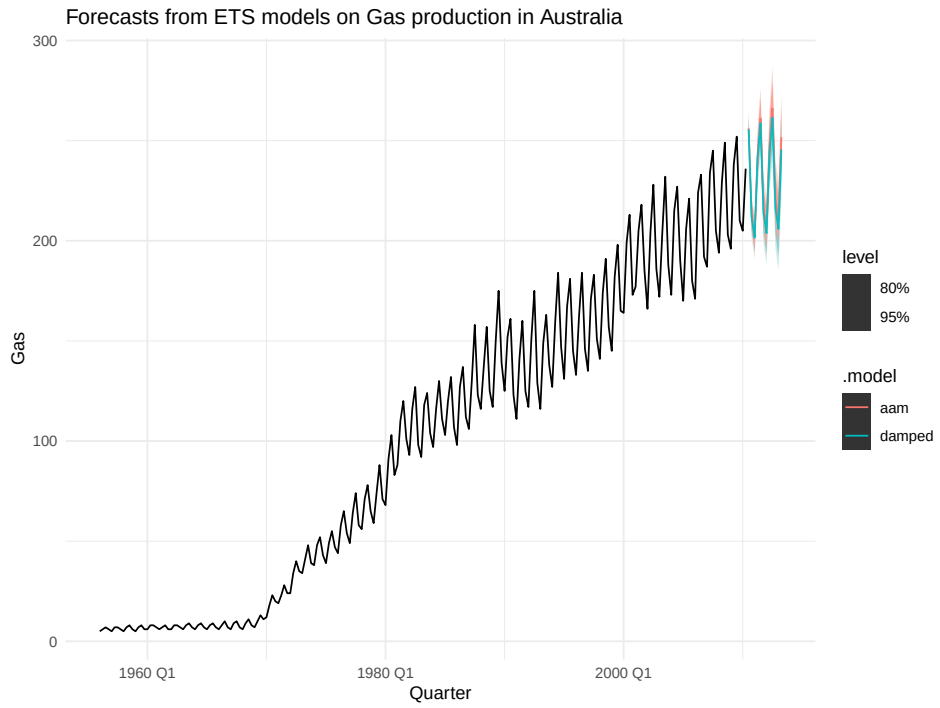
Multiplicative seasonality is needed because the seasonality itself is not "linear", in that the range of variances changes over time (increasing, in this series).

The damped trend improves the forecast, slightly taming the overall trajectory of the series and narrowing the prediction interval.

```
gas <- aus_production |>
  select(Gas)

# Check the data
autoplot(gas)
```




```
# Fit the model
gas_fit <- gas |>
  model(aam = ETS(Gas ~ error("A") + trend("A") + season("M")),
        damped = ETS(Gas ~ error("A") + trend("Ad") + season("M"))
        )

gas_fc <- gas_fit |>
  forecast(h = "3 years")

gas_fc |>
  autoplot(gas) +
  labs(title = "Forecasts from ETS models on Gas production in Australia")
```

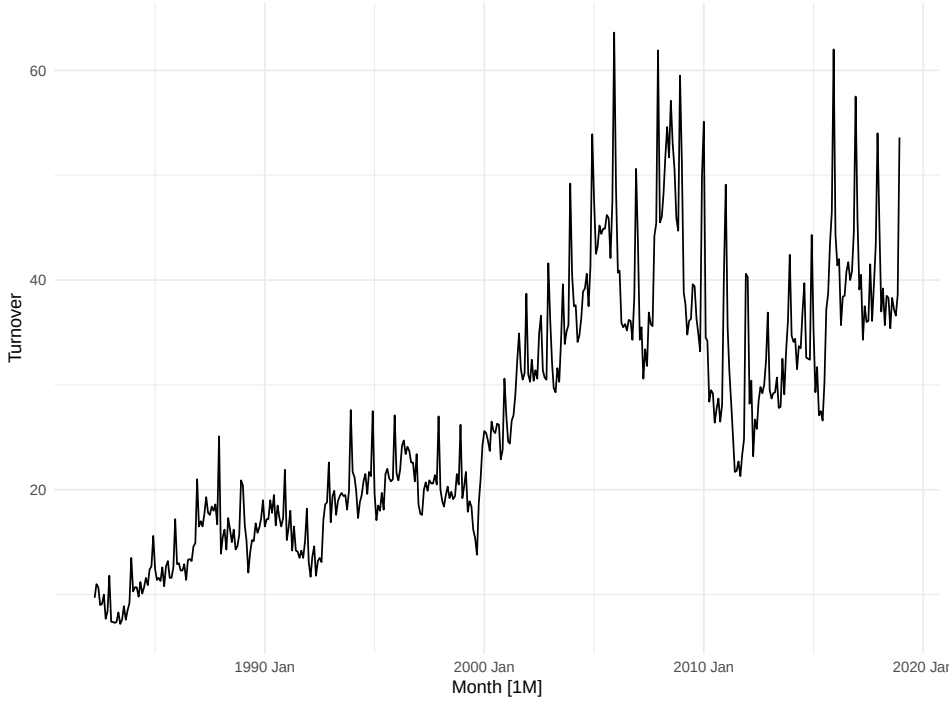


8.8

Recall your retail time series data (from Exercise 7 in Section 2.10).

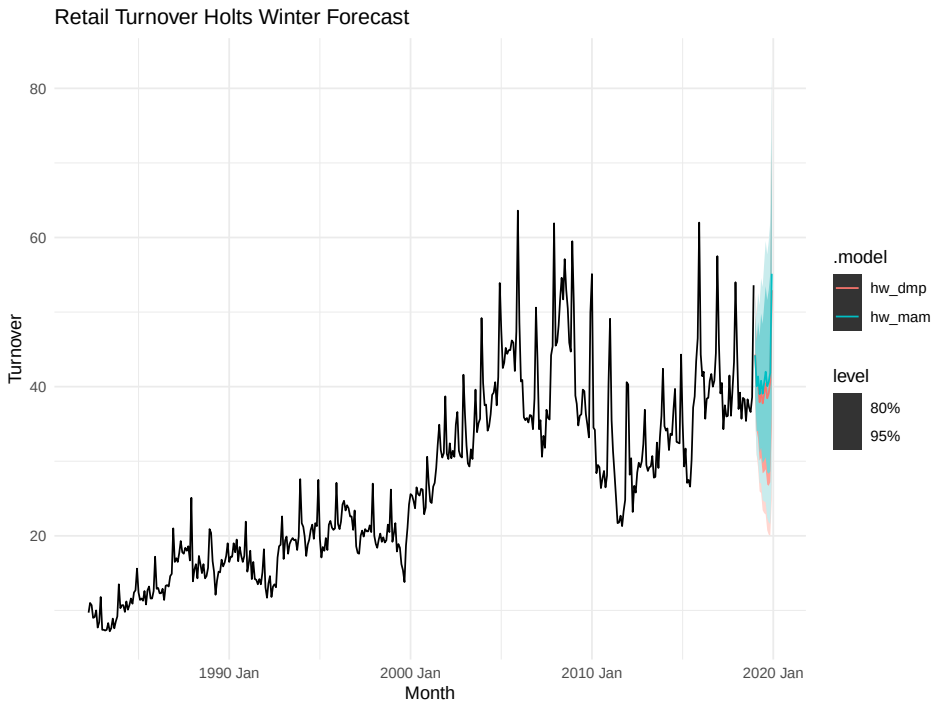
- Why is multiplicative seasonality necessary for this series?
- Apply Holt-Winters' multiplicative method to the data. Experiment with making the trend damped.
- Compare the RMSE of the one-step forecasts from the two methods. Which do you prefer?
- Check that the residuals from the best method look like white noise.
- Now find the test set RMSE, while training the model to the end of 2010. Can you beat the seasonal naïve approach from Exercise 7 in Section 5.11?

```
# Set up data
set.seed(42)

myseries <- aus_retail |>
  filter(`Series ID` == sample(aus_retail$`Series ID`,1))

autoplot(myseries)
```



Multiplicative seasonality is necessary because the seasonal variance increases over time.

```
# Apply Holt-Winter's MAM

myseries_fit <- myseries |>
  model(
    hw_mam = ETS(Turnover ~ error("M") + trend("A") + season("M")),
    hw_dmp = ETS(Turnover ~ error("M") + trend("Ad") + season("M"))
)

myseries_fc <- myseries_fit |>
  forecast(h=12)

myseries_fc |>
  autoplot(myseries) +
  labs(title="Retail Turnover Holts Winter Forecast")
```



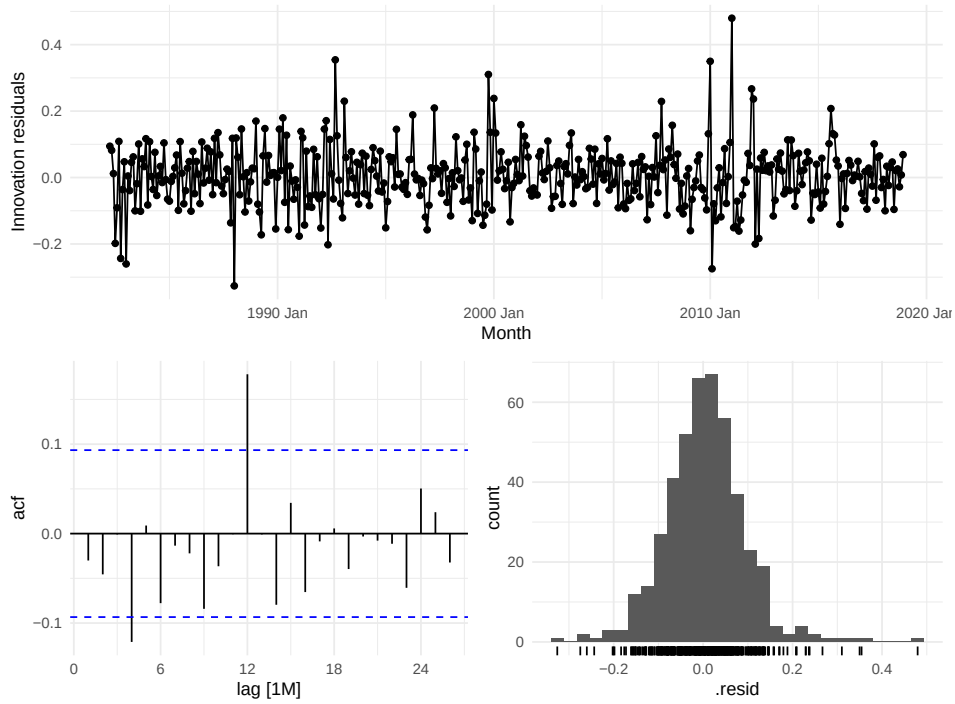
The RMSE of the damped and regular Holt-Winters' methods are nearly identical (for a single forecast). This is expected, as damping will strengthen over longer forecasts, but for a single step forecast the effect is minimal. I would have no real preference between the two, especially as it is unrealistic to expect a single step forecast in a real world scenario.

```
# One step forecast RMSE
myseries_fit %>%
  accuracy() %>%
  select(.model, RMSE)
```

```
# A tibble: 2 x 2
  .model  RMSE
  <chr>  <dbl>
1 hw_mam  2.54
2 hw_dmp  2.54
```

Plotting the residuals, we can see high homoscedasticity and no autocorrelation. The residuals are also normally distributed about zero, which gives me confidence the residuals are just noise.

```
# Plot residuals

myseries_fit |>
  select(hw_dmp) |>
  gg_tsresiduals()
```



Analyzing RMSE only, our Holt-Winters' with damping narrowly beats out the seasonal naive method. The difference is slim and the computational performance may favor the naive method in some production environments.

```
# Test train split
train <- myseries %>% filter(year(Month) < 2010)
test <- myseries %>% filter(year(Month) >= 2010)

# Fit on train set
fit <- train %>%
  model(
    hw_mam = ETS(Turnover ~ error("M") + trend("A") + season("M")),
    hw_dmp = ETS(Turnover ~ error("M") + trend("Ad") + season("M")),
    snaive = SNAIVE(Turnover)
```

```
  )

# Forecast on test set
fc <- fit %>% forecast(h = nrow(test))

# Check accuracy
fc |> accuracy(test)
```

```
# A tibble: 3 x 12
  .model State    Industry .type    ME  RMSE   MAE   MPE  MAPE  MASE RMSSE  ACF1
  <chr>  <chr>    <chr>    <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 hw_dmp Western~ Newspap~ Test  -2.32  6.88  5.71 -10.7  18.1   NaN   NaN 0.756
2 hw_mam Western~ Newspap~ Test  -9.19 10.5   9.70 -29.5  30.5   NaN   NaN 0.556
3 snaive Western~ Newspap~ Test  -3.91  7.79  6.35 -14.9  20.4   NaN   NaN 0.725
```

8.9

For the same retail data, try an STL decomposition applied to the Box-Cox transformed series, followed by ETS on the seasonally adjusted data. How does that compare with your best previous forecasts on the test set?

The STL Box Cox plus seasonally adjusted ETS model performs second best, just behind Holt-Winters with damping. The difference, again is quite small, so the domain context would need to make the decision on which we would choose.

```
 # Test train split

lambda <- myseries %>%
  features(Turnover, features = guerrero) %>%
  pull(lambda_guerrero)

# Fit on train set
fit <- train %>%
  model(
    hw_mam = ETS(Turnover ~ error("M") + trend("A") + season("M")),
    hw_dmp = ETS(Turnover ~ error("M") + trend("Ad") + season("M")),
    snaive = SNAIVE(Turnover),
    STL = decomposition_model(
      STL(box_cox(Turnover, lambda = lambda)),
      ETS(season_adjust ~ season("N"))
    )
```

```
  )

# Forecast on test set
fc <- fit %>% forecast(h = nrow(test))

# Check accuracy
fc |> accuracy(test)
```

```
# A tibble: 4 x 12
  .model State    Industry .type    ME  RMSE   MAE   MPE  MAPE  MASE RMSSE  ACF1
  <chr>  <chr>    <chr>    <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 STL    Western~ Newspap~ Test  -4.68  7.06  5.73 -16.6  18.7   NaN   NaN 0.663
2 hw_dmp Western~ Newspap~ Test  -2.32  6.88  5.71 -10.7  18.1   NaN   NaN 0.756
3 hw_mam Western~ Newspap~ Test  -9.19 10.5   9.70 -29.5  30.5   NaN   NaN 0.556
4 snaive Western~ Newspap~ Test  -3.91  7.79  6.35 -14.9  20.4   NaN   NaN 0.725
```